

LCNCM — Liability Condition Continuity Module

LCNCM — Liability Condition Continuity Module

Architecture: LIGA™ — Liability Governance Architecture
Parent Standard: Liability Conditions Standard (LCOS)
Category: Governance & Enforcement
Subcategory: Liability Governance
Type: Core Module
Governed Space: Liability Condition Continuity
Module ID: LIGA-LCOS-LCNCM-001
Origin ID: OOF.CM.LIGA.05.05.01
Version: 1.0
Status: Canonical · Open Standard
Effective Date: 22 September 2026
Compatibility: OOF® Methodology OS™ · Modular OOF® Architecture Ecosystem
AI-Readable: Yes
Authority: OOF® — OriginOpen® Foundation
Protection: MIP® — Methodological Intellectual Property
Canonical Language: English (UCL™)

Canonical Definition

The Liability Condition Continuity Module (LCNCM) governs how Liability Conditions and their Requirements, States, associations, Modifiers, changes, and historical representations remain identifiable, traceable, reconstructable, and temporally coherent throughout the Liability lifecycle.

LCNCM preserves the governed lineage of Liability Conditions while allowing legitimate changes in Requirement, State, applicability, scope, association, Exception, Exclusion, Waiver, limitation, and other Condition-relevant structures.

It governs the transition:

Exception-Governed Liability Condition State Set → Condition Continuity Establishment → Requirement & State Lineage → Modifier & Change Preservation → Historical Reconstruction → Governed Liability Condition

Structure

Its foundational distinction is:

Liability Condition Continuity ≠ Liability Condition Immutability.

Operational Role

The operational role of LCNCM is:

to preserve the identity, lineage, temporal coherence, and reconstructability of the complete governed Liability Condition structure as Conditions and their associated states change throughout the Liability lifecycle.

LCNIM identifies the Conditions.

LCNRM defines what they require.

LCNSM determines their States.

LCNXM governs their Exceptions, Exclusions, Waivers, and other Modifiers.

LCNCM preserves how all of those states relate across time.

Position in the Parent Standard Flow

The internal LCOS progression is:

Liability Condition Identification → Liability Condition Requirement → Liability Condition State → Liability Condition Exception → Liability Condition Continuity LCNCM governs:

Liability Condition Continuity

Its principal input is:

Exception-Governed Liability Condition State Set

Its principal output is:

Governed Liability Condition Structure

Its canonical handoff is:

Governed Liability Condition Structure → Liability Connection Governance

Central Operational Question

How can Liability Conditions, their Requirements, States, associations, and Modifiers remain identifiable, traceable, historically coherent, and reconstructable while allowing legitimate change throughout the Liability lifecycle?

Governed Space

LCNCM governs:

Condition Identity Continuity, Condition Requirement Continuity, Condition State Lineage, Condition Association Continuity, Condition Modifier Continuity, Condition Versioning, Condition Change, Condition Correction, Condition Reassessment, Condition Supersession, Condition Activation, Condition Suspension, Condition Termination, Temporal Continuity, Historical Condition State, Condition Reconstruction, Cross-System Continuity, Lifecycle Re-Entry, Condition Set Evolution, and Condition Continuity Sufficiency.

Continuity Principle

Continuity means that a Liability Condition can remain governably connected to its origin and historical states despite legitimate change.

Continuity does not require the Condition to remain unchanged.

Therefore:

Continuity ≠ Immutability.

Condition Identity Continuity

A Condition should retain persistent identity while changes occur that do not materially create a different Condition.

This permits the system to distinguish:

one Condition changing over time, from:

multiple distinct Conditions.

Therefore:

Condition Change ≠ New Condition by default.

Condition Origin

The origin of a Condition should remain traceable throughout its lifecycle.

Origin may include:

the Liability Basis that made it relevant, its original identification context, relevant Subject association, Relationship association, Object association, and temporal origin.

Later changes should not erase that origin.

Current Condition and Origin

A Condition's current representation may differ substantially from its original representation.

Therefore:

Current Condition State $\neq$ Condition Origin.

Both must remain reconstructable.

Condition Identity and Representation

A Condition may be represented differently across:

systems, documents, interfaces, versions, or governance stages.

A representational change does not necessarily create a new Condition.

Therefore:

Representation Change $\neq$ Condition Identity Change by default.

Condition Identity and Label

A Condition may receive a revised label or description.

Where the underlying governed Condition remains the same:

Condition Label Change $\neq$ New Condition.

The earlier label should remain traceable where materially relevant.

Condition Requirement Continuity

The Requirement associated with a Condition may change.

LCNCM preserves:

earlier Requirement, later Requirement, effective time, reason for change, and relationship between versions.

Therefore:

Requirement Change $\neq$ New Liability Condition by default.

Requirement Versioning

Material Requirement changes should be versioned where necessary for historical reconstruction.

The system should be able to answer:

Which Requirement applied to this Condition at the relevant time?

Therefore:

Current Requirement Version $\neq$ Historically Applicable Requirement Version by default.

Requirement Supersession

A later Requirement may supersede an earlier Requirement.

Supersession changes which Requirement governs prospectively or for a defined period.

It does not erase the earlier Requirement.

Therefore:

Requirement Supersession $\neq$ Historical Deletion.

Partial Requirement Change

Only part of a Requirement may change.

The unchanged components should preserve continuity.

Therefore:

Partial Requirement Change $\neq$ Complete Requirement Replacement by default.

Requirement Correction

An earlier Requirement representation may have been incorrect.

Correction should preserve:

the incorrect representation, corrected representation, correction time, effective relevance, reason, and supporting source.

Therefore:

Requirement Correction $\neq$ Historical Erasure.

Condition State Lineage

LCNCM preserves the sequence of governed Condition States across time.

Example:

Not Yet Evaluated $\rightarrow$ Pending $\rightarrow$ Unknown $\rightarrow$ Satisfied $\rightarrow$ Not Satisfied Each materially relevant transition should remain reconstructable.

Current State and Historical State

The current Condition State does not replace earlier States.

Therefore:

Current Condition State $\neq$ Complete Condition State History.

Historical States may remain essential to later Liability governance.

State Change

A State Change occurs where the underlying governed reality or applicable conditional structure legitimately changes.

Example:

Pending $\rightarrow$ Satisfied or:

Satisfied $\rightarrow$ Not Satisfied State Change should remain distinct from correction.

State Correction

A State Correction occurs where the earlier State representation was incorrect rather than where reality itself changed.

Therefore:

Condition State Correction $\neq$ Condition State Change.

Both the earlier and corrected representations should remain traceable.

State Reassessment

A Condition State may require reassessment because of:

new evidence, newly validated information, Requirement clarification, Modifier discovery, historical reconstruction, or lifecycle re-entry.

Therefore:

Condition State Reassessment $\neq$ Historical Rewrite.

Later Knowledge

Information discovered at T2 may alter what is known about a Condition at T1.

The architecture must distinguish:

state existing at T1, knowledge available at T1, and knowledge discovered at T2.

Therefore:

Later Knowledge $\neq$ Earlier Knowledge.

Retroactive Reassessment

A later discovery may require reassessment of an earlier Condition State.

Where this occurs, the architecture should preserve:

original State determination, information originally available, later information, reassessed State, and reason for reassessment.

Therefore:

Retroactive Reassessment $\neq$ Historical Record Erasure.

Condition Association Continuity

A Condition may be associated with:

Liability Basis, Subject, Relationship, Object, or several such structures.

Those associations may change.

LCNCM preserves their historical states.

Basis Association Change

A Condition may cease to operate under one Liability Basis or become relevant under another.

Therefore:

Basis Association Change $\neq$ Condition Identity Change by default.

Where the change materially creates a distinct Condition, separate identity may be required.

Subject Association Change

A Condition may become relevant to:

a new Subject, fewer Subjects, or a changed Subject role.

Therefore:

Subject Association Change $\neq$ Condition Identity Change by default.

Historical Subject associations should remain preserved.

Relationship Association Change

A Condition may move into or out of relevance within a particular Liability Relationship.

Therefore:

Relationship Association Change $\neq$ Condition Identity Change by default.

Object Association Change

A Condition may change its association with a Governed Liability Object or Object component.

Therefore:

Object Association Change $\neq$ Condition Identity Change by default.

The system should preserve which Object association applied at each relevant time.

Association Change and Liability

A changed Condition association does not automatically create or eliminate Liability.

Therefore:

Condition Association Change $\neq$ Liability Change by default.

Downstream governance may require reassessment.

Modifier Continuity

Exceptions, Exclusions, Waivers, exemptions, limitations, and other Modifiers may:

activate, change, expand, contract, suspend, expire, terminate, or be reassessed.

LCNCM preserves their lineage.

Modifier Activation History

The system should preserve when a Modifier became applicable.

Therefore:

Current Modifier Applicability $\neq$ Historical Modifier Applicability by default.

Modifier Expiration

An expired Modifier remains part of the historical Condition structure.

Therefore:

Modifier Expiration $\neq$ Modifier Erasure.

Modifier Suspension

A suspended Modifier retains identity unless governance establishes otherwise.

Therefore:

Modifier Suspension $\neq$ Modifier Termination.

Modifier Scope Change

The scope of a Modifier may expand or contract.

LCNCM should preserve:

earlier scope, later scope, effective time, and reason for change.

Therefore:

Modifier Scope Change $\neq$ New Modifier by default.

Modifier Correction

An incorrectly represented Modifier may be corrected.

Therefore:

Modifier Correction $\neq$ Historical Deletion.

The correction should remain linked to the original representation.

Condition Activation

A Condition may exist before becoming operationally applicable.

The architecture should distinguish:

Condition existence, Condition activation, and Condition State.

Therefore:

Condition Existence $\neq$ Condition Activation.

Condition Suspension

A Condition may temporarily cease active operation without losing identity.

Therefore:

Condition Suspension $\neq$ Condition Termination.

Condition Reactivation

A suspended or inactive Condition may later reactivate.

Reactivation does not necessarily create a new Condition.

Therefore:

Condition Reactivation ≠ New Condition by default.

Condition Termination

A Condition may cease to operate within the active Liability structure.

Termination should preserve:

Condition Identity, historical Requirements, States, Modifiers, associations, and termination reason.

Therefore:

Condition Termination ≠ Condition Erasure.

Condition Supersession

One Condition may be superseded by another where governance establishes a legitimate successor structure.

LCNCM should preserve:

predecessor, successor, effective transition, and relationship between them.

Therefore:

Condition Supersession ≠ Historical Deletion.

Successor Condition

A Successor Condition is not necessarily a new version of the same Condition.

The architecture should distinguish:

Condition Version

from:

Successor Condition.

Therefore:

Successor Condition ≠ Condition Version by default.

Condition Split

One Condition may later require separation into multiple distinct Conditions.

The system should preserve the common origin.

Therefore:

Condition Split ≠ Independent Historical Origins.

Condition Consolidation

Several Conditions may later be consolidated where governance establishes that a unified structure is appropriate.

The earlier Conditions should remain reconstructable.

Therefore:

Condition Consolidation ≠ Historical Identity Erasure.

Duplicate Reconciliation

Multiple records may be discovered to represent the same underlying Condition.

Reconciliation should preserve the provenance of the duplicate records.

Therefore:

Record Duplication ≠ Condition Duplication.

Condition Set Evolution

The complete Condition Set may evolve as Conditions are:

identified, activated, changed, suspended, terminated, split, consolidated, or newly discovered.

Therefore:

Current Condition Set $\neq$ Complete Historical Condition Set.

Condition Set Expansion

New Conditions may enter the governed structure.

Expansion may result from:

new information, new Basis relevance, new Subjects, new Relationships, new Objects, or lifecycle re-entry.

Therefore:

Condition Set Expansion $\neq$ New Liability Matter by default.

Condition Set Contraction

Conditions may leave the active set through:

termination, supersession, non-applicability, or another governed transition.

Therefore:

Condition Set Contraction $\neq$ Historical Condition Erasure.

Temporal Continuity

LCNCM should preserve the temporal relationship among:

Condition identification, Requirement applicability, Condition State, Modifier applicability, association changes, and downstream lifecycle events.

This enables reconstruction of the Condition structure at any materially relevant time.

Condition State at T1 and T2

The architecture should support:

Condition Structure at T1

and:

Condition Structure at T2

without assuming that either represents the other.

Therefore:

Condition Structure at T2 $\neq$ Condition Structure at T1 by default.

Historical Snapshot

A Historical Condition Snapshot represents the governed Condition structure at a defined lifecycle moment.

It may include:

Condition Set, Requirements, States, associations, Modifiers, uncertainties, and applicable versions.

A snapshot does not replace the full lineage.

Historical Reconstruction

LCNCM should enable reconstruction of:

which Conditions existed, which Requirements applied, which States were established, which Modifiers applied, which Subjects, Relationships, Objects, and Bases were associated, and what uncertainty existed at a materially relevant

time.

Current State and Reconstruction

A current database state is insufficient where earlier Liability governance depended upon historical Conditions.

Therefore:

Current Condition Record $\neq$ Historical Condition Reconstruction.

Reconstruction and Later Knowledge

Historical reconstruction should distinguish:

what actually existed, what was recorded, what was known, and what was discovered later.

These are not necessarily identical.

Provenance Continuity

Condition lineage should preserve provenance sufficient to establish where material representations originated and how they changed.

Provenance supports reconstructability without replacing substantive Condition governance.

Cross-System Continuity

Liability Condition records may move across:

information systems, organizations, service providers, administrative structures, or technical environments.

Such movement should not automatically create a new Condition identity.

Therefore:

System Migration $\neq$ New Liability Condition.

Multiple System Records

The same Condition may appear in multiple systems.

Therefore:

Multiple System Records $\neq$ Multiple Liability Conditions by default.

Persistent identity and provenance should support reconciliation.

System Identifier Change

A technical identifier may change during migration.

Where Condition identity remains continuous:

System Identifier Change $\neq$ Condition Identity Change.

Origin and lineage should remain preserved.

Organizational Change

An organization responsible for administering Condition records may change.

Therefore:

Administrative Ownership Change $\neq$ Liability Condition Change by default.

Governance ownership of the record and identity of the Condition remain distinct.

Cross-Architecture Continuity

A Condition may depend on states governed by other OOF® architectures.

Where those external states change, LCNCM should preserve which external state was used at each materially relevant lifecycle moment.

It does not take ownership of that external state.

External State Change

A change in an external governed state may trigger Condition reassessment.

However:

External State Change $\neq$ Automatic Condition State Change.

LCNSM may need to re-evaluate the Condition against its Requirement.

Liability Basis Change

A Liability Basis may change while the Condition lifecycle remains active.

LCNCM should determine whether the change requires:

continued Condition identity, Requirement revision, Condition reassessment, Condition termination, new Condition formation, or lifecycle re-entry.

Therefore:

Liability Basis Change $\neq$ Automatic New Condition.

Liability Relationship Change

A Liability Relationship may change while a Condition remains relevant.

Therefore:

Liability Relationship Change $\neq$ Automatic Condition Termination.

The affected association and Condition relevance should be reassessed.

Liability Object Change

A Governed Liability Object may change.

This may affect Condition relevance or Requirement.

Therefore:

Liability Object Change $\neq$ Automatic Condition Identity Change.

The earliest materially affected governance space should be revisited.

Lifecycle Re-Entry

Later LIGA™ stages may reveal:

a missing Condition, incorrect Requirement, incorrect State, missed Modifier, incorrect association, or historical inconsistency.

LCNCM supports controlled re-entry to the earliest materially affected LCOS governance stage.

Targeted Re-Entry

Re-entry should be targeted.

Examples:

missing Condition $\rightarrow$ LCNIM incorrect Requirement $\rightarrow$ LCNRM incorrect State $\rightarrow$ LCNSM missed Exception $\rightarrow$ LCNXM continuity defect $\rightarrow$ LCNCM Therefore:

Lifecycle Re-Entry $\neq$ Full LCOS Restart by default.

Re-Entry and Origin

Re-entering an earlier governance stage does not automatically create a new Condition Origin.

Therefore:

Re-Entry $\neq$ New Origin by default.

The historical lineage should remain continuous unless governance establishes a genuinely new Condition.

Re-Entry and History

Re-entry must preserve earlier governance states.

Therefore:

Re-Entry ≠ Historical Rewrite.

A later corrected or expanded structure should remain linked to what was previously governed.

Downstream Reassessment

A Condition change may require reassessment of downstream LIGA™ stages.

Potentially affected spaces include:

Liability Connection, Liability Determination, Liability Scope & Allocation, Liability State & Change, Transfer & Recourse, Resolution & Discharge, and Continuity & Finality.

The Condition change itself does not determine those downstream outcomes.

Condition Change and Connection

A changed Condition State or Modifier may affect Liability Connection governance.

Therefore:

Condition Change ≠ Connection Change by default.

LCNS determines whether the Connection is affected.

Condition Change and Liability Determination

A Condition change may require Liability Determination reassessment.

However:

Condition Change ≠ Liability Determination Change by default.

LDS owns Determination.

Condition Change and Liability State

Even after Liability has been determined, a later Condition change or historical correction may affect the active Liability lifecycle.

Therefore:

Condition State Change ≠ Liability State Change by default.

The relevant downstream stage must govern the consequence.

Resolution and Condition History

Liability Resolution does not erase the Conditions upon which earlier governance depended.

Therefore:

Liability Resolution ≠ Condition History Erasure.

Condition lineage remains relevant to reconstructability.

Finality and Condition History

Liability Finality does not erase historical Conditions.

Therefore:

Liability Finality ≠ Liability Condition Erasure.

Finality ends the relevant active Liability governance state according to LCFS; it does not destroy the historical basis of that state.

Continuity and Immutability

LCNCM does not require historical records to remain technically unchangeable.

It requires legitimate changes to be:

governed, traceable, attributable, versioned where necessary, and reconstructable.

Therefore:

Governed Change $\neq$ Loss of Continuity.

Continuity and Completeness

Condition continuity does not require storing every technically observable change.

It requires preservation of changes materially necessary to reconstruct the governed Liability Condition lifecycle.

Therefore:

Maximum Record Volume $\neq$ Maximum Governance Quality.

Continuity Gap

A Continuity Gap exists where the Condition lineage cannot reliably explain a materially relevant transition.

Examples may include:

unknown Requirement change, missing State transition, untraceable Modifier activation, unexplained association change, or missing historical version.

A Continuity Gap should remain explicit until resolved.

Continuity Conflict

Different records may provide incompatible histories.

LCNCM should preserve the conflict and available provenance.

Therefore:

Continuity Conflict $\neq$ Authority to Manufacture Historical Certainty.

Continuity Uncertainty

Where the historical Condition structure cannot be fully reconstructed, uncertainty should be explicitly represented.

Therefore:

Incomplete History $\neq$ Authority to Infer Complete History.

Continuity Sufficiency

Condition continuity is sufficient when the governed structure can reliably establish, for materially relevant lifecycle moments:

which Condition existed, which Requirement applied, which State existed, which Modifiers applied, which Basis, Subjects, Relationships, and Objects were associated, what changed, when it changed, why it changed, and what uncertainty remains.

Readiness for Liability Connection Governance

LCOS is ready to hand off to LCNS when the materially relevant Condition structure is sufficiently:

identified, defined, evaluated, modifier-governed, temporally coherent, and reconstructable.

This does not require every Condition to be Satisfied.

Therefore:

Condition Governance Readiness $\neq$ Condition Satisfaction.

Governed Liability Condition Structure

The principal LCOS output is:

Governed Liability Condition Structure

It represents the governed conditional structure relevant to the potential Liability inquiry, including materially necessary:

Condition identities, Requirements, States, associations, Modifiers, temporal states, changes, uncertainties, and lineage.

It is an input to Liability Connection governance.

It is not Liability.

Governed Condition Structure and Connection

The Governed Liability Condition Structure allows LCNS to determine whether the required Liability-Relevant Connection exists.

Therefore:

Governed Liability Condition Structure $\neq$ Liability-Relevant Connection.

Governed Condition Structure and Determination

Even a complete, stable, and fully reconstructable Condition Structure does not establish Liability.

Therefore:

Governed Liability Condition Structure $\neq$ Liability Determination.

The canonical progression remains:

Basis $\rightarrow$ Relationships $\rightarrow$ Conditions $\rightarrow$ Connection $\rightarrow$ Determination

Minimum Implementation Framework

A conforming implementation of LCNCM must provide at least the following eight capabilities:

Persistent Condition Identity Governance

Capability to preserve Condition identity across representation changes, labels, systems, Requirement changes, State changes, and other legitimate lifecycle transitions.

Requirement, State & Modifier Lineage

Capability to preserve the historical lineage of Condition Requirements, Condition States, Exceptions, Exclusions, Waivers, exemptions, limitations, and other materially relevant Modifiers.

Association & Temporal Continuity

Capability to preserve changing associations between Conditions and Liability Bases, Subjects, Relationships, Objects, and relevant temporal states.

Change, Version, Supersession & Successor Governance

Capability to distinguish change, versioning, partial change, supersession, successor Conditions, splitting, consolidation, activation, suspension, termination, and reactivation without inappropriate identity loss.

Correction, Reassessment & Lifecycle Re-Entry

Capability to preserve corrections, reassessments, later knowledge, retroactive review, and targeted lifecycle re-entry without historical rewriting or unnecessary lifecycle restart.

Cross-System & Cross-Architecture Continuity

Capability to preserve Condition identity and lineage across system migration, multiple records, organizational change, and external governed-state dependencies.

Historical Reconstruction

Capability to reconstruct materially relevant historical Condition structures, Requirements, States, Modifiers, associations, changes, knowledge states, and unresolved uncertainty at defined lifecycle moments.

Continuity Sufficiency & Downstream Handoff

Capability to determine whether the Liability Condition structure is sufficiently coherent and reconstructable for handoff to Liability Connection governance.

Governance Logic

The canonical LCNCM governance logic is:

Exception-Governed Liability Condition State Set

↓

Establish Persistent Condition Identity

↓

Preserve Condition Origin

↓

Preserve Requirement Lineage

↓

Preserve Condition State Lineage

↓

Preserve Modifier Lineage

↓

Preserve Basis / Subject / Relationship / Object Associations

↓

Preserve Temporal Applicability

↓

Govern Change / Version / Supersession / Successor State

↓

Govern Activation / Suspension / Termination / Reactivation

↓

Preserve Correction / Reassessment

↓

Reconcile Cross-System Records

↓

Govern Lifecycle Re-Entry

↓

Preserve Historical Reconstruction

↓

Identify Continuity Gaps / Conflicts / Uncertainty

↓

Assess Continuity Sufficiency

↓

Governed Liability Condition Structure

↓

Handoff to Liability Connection Governance

Governance Outputs

LCNCM may produce:

Persistent Liability Condition Identity

Condition Origin Record

Condition Requirement Lineage

Condition State Lineage

Condition Modifier Lineage

Condition Association History

Condition Temporal History

Condition Version Record

Condition Change Record

Condition Correction Record

Condition Reassessment Record

Condition Supersession Record

Condition Successor Record

Condition Split Record

Condition Consolidation Record

Condition Activation Record

Condition Suspension Record

Condition Reactivation Record

Condition Termination Record

Condition Set Evolution Record

Historical Condition Snapshot

Condition Reconstruction Record

Cross-System Continuity Record

Lifecycle Re-Entry Record

Continuity Gap

Continuity Conflict

Continuity Uncertainty State

Condition Continuity Sufficiency State

The principal output is:

Governed Liability Condition Structure

Critical Distinctions

LCNCM preserves the following distinctions:

Liability Condition Continuity ≠ Liability Condition Immutability Continuity ≠ Immutability Condition Change ≠ New Condition by default Current Condition State ≠ Condition Origin Representation Change ≠ Condition Identity Change by default Condition Label Change ≠ New Condition Requirement Change ≠ New Liability Condition by default Current Requirement Version ≠ Historically Applicable Requirement Version by default Requirement Supersession ≠ Historical Deletion Partial Requirement Change ≠ Complete Requirement Replacement by default Requirement Correction ≠ Historical Erasure Current Condition State ≠ Complete Condition State History Condition State Correction ≠ Condition State Change Condition State Reassessment ≠ Historical Rewrite Later Knowledge ≠ Earlier Knowledge Retroactive Reassessment ≠ Historical Record Erasure Basis Association Change ≠ Condition Identity Change by default Subject Association Change ≠ Condition Identity Change by default Relationship Association Change ≠ Condition Identity Change by default Object Association Change ≠ Condition Identity Change by default Condition Association Change ≠ Liability Change by default Current Modifier Applicability ≠ Historical Modifier Applicability by default Modifier Expiration ≠ Modifier Erasure Modifier Suspension ≠ Modifier Termination Modifier Scope Change ≠ New Modifier by default Modifier Correction ≠ Historical Deletion Condition Existence ≠ Condition Activation Condition Suspension ≠ Condition Termination Condition Reactivation ≠ New Condition by default Condition Termination ≠ Condition Erasure Condition Supersession ≠ Historical Deletion Successor Condition ≠ Condition Version by default Condition Split ≠ Independent Historical Origins Condition Consolidation ≠ Historical Identity Erasure Record Duplication ≠ Condition Duplication Current Condition Set ≠ Complete Historical Condition Set Condition Set Expansion ≠ New Liability Matter by default Condition Set Contraction ≠ Historical Condition Erasure Condition Structure at T2 ≠ Condition Structure at T1 by default Current Condition Record ≠ Historical Condition Reconstruction System Migration ≠ New Liability Condition Multiple System Records ≠ Multiple Liability Conditions by default System Identifier Change ≠ Condition Identity Change Administrative Ownership Change ≠ Liability Condition Change by default External State Change ≠ Automatic Condition State Change Liability Basis Change ≠ Automatic New Condition Liability Relationship Change ≠ Automatic Condition Termination Liability Object Change ≠ Automatic Condition Identity Change Lifecycle Re-Entry ≠ Full LCOS Restart by default Re-Entry ≠ New Origin by default Re-Entry ≠ Historical Rewrite Condition Change ≠ Connection Change by default Condition Change ≠ Liability Determination Change by default Condition State Change ≠ Liability State Change by default Liability Resolution ≠ Condition History Erasure Liability Finality ≠ Liability Condition Erasure Governed Change ≠ Loss of Continuity Maximum Record Volume ≠ Maximum Governance Quality Continuity Conflict ≠ Authority to Manufacture Historical Certainty Incomplete History ≠ Authority to Infer Complete History Condition Governance Readiness ≠ Condition Satisfaction Governed Liability Condition Structure ≠ Liability-Relevant Connection Governed Liability Condition Structure ≠ Liability Determination

Cross-Architecture Boundary

LCNCM may preserve references to governed states originating in other OOF® architectures where those states materially affect the Liability Condition lifecycle.

GOA™ may provide Authority, Scope, and Delegation histories.

ORA™ may provide historical Operational Reality.

AGA™ may provide Responsibility, Accountability, Authority, and Attribution states across time.

HAGA™ may provide governed Harm states and Harm lifecycle changes.

ADIT® may provide evidence lineage and auditability.

VALIDOS™ may provide validation states and validation history.

OBIDENTITY® may provide Identity, Ownership, Authority, Permission, Provenance, Transfer, and Continuity states.

AIG® and ASGA™ may provide AI or autonomous-system governance states relevant to historical Condition reconstruction.

LCNCM preserves the Liability-specific relationship to those states without taking governance ownership of their original domains.

The canonical interoperability principle is:

An externally governed state may change, expire, be corrected, or be superseded while LIGA™ preserves only its Liability-conditional relevance and historical relationship to the Liability Condition structure.

Canonical Boundary

LCNCM begins with an:

Exception-Governed Liability Condition State Set

and asks:

How can the complete governed Condition structure remain identifiable and reconstructable as its Requirements, States, associations, and Modifiers change throughout the Liability lifecycle?

LCNCM governs:

Condition Identity Continuity, Condition Origin, Requirement Lineage, State Lineage, Modifier Lineage, Association Continuity, Temporal Continuity, Versioning, Change, Correction, Reassessment, Supersession, Successor Conditions, Condition Splitting, Condition Consolidation, Activation, Suspension, Reactivation, Termination, Condition Set Evolution, Cross-System Continuity, Cross-Architecture References, Historical Reconstruction, Lifecycle Re-Entry, Continuity Gaps, Continuity Conflicts, Continuity Uncertainty, and Continuity Sufficiency.

LCNCM does not determine:

new Condition identity except where continuity analysis establishes that a materially distinct Condition must return to LCNCM, substantive Requirement formation except through re-entry to LCNCM, Condition State except through re-entry to LCNCM, Exception or Exclusion applicability except through re-entry to LCNCM, Liability-Relevant Connection, Liability Attachment, Liability Determination, Liability Scope, Liability Allocation, Liability State, Liability Transfer, Recourse, Resolution, Discharge, or Finality.

LCNCM ends when a:

Governed Liability Condition Structure

is sufficiently coherent, temporally governed, historically reconstructable, and ready for Liability Connection governance.

Use Case 1 — AI Service Condition Evolution

An AI-enabled service initially requires a defined human-oversight Condition.

The Condition is identified, its Requirement is established, and its State is evaluated.

Later, the operational model changes, a revised Requirement becomes applicable, and a limited Exception is introduced for a specific automated process.

LCNCM preserves:

the original Condition identity, the original Requirement, the revised Requirement, the Condition States under each relevant period, the Exception's activation and scope, and the relationship between the historical and current structures.

The current configuration does not erase the Condition structure that governed earlier AI operations.

The resulting history remains reconstructable for later Liability Connection and Determination governance.

Use Case 2 — Industrial Maintenance Condition History

An industrial equipment Condition requires maintenance within a defined interval.

The equipment later changes operator, the maintenance Relationship changes, and a Requirement is revised for a later operating period.

A historical review subsequently discovers that an earlier maintenance record had been represented incorrectly.

LCNCM preserves:

the Condition's persistent identity, the earlier and later Requirements, the changing Subject and Relationship associations, the original Condition State, the corrected historical State, the reason for correction, and the applicable temporal boundaries.

The correction does not rewrite the earlier governance record.

It creates a traceable corrected lineage that can be used by subsequent Liability Connection governance.

Canonical Closing Statement

Liability Conditions do not remain static merely because they have already been identified and evaluated.

Requirements change. States change. Relationships change. Modifiers activate and expire. New information may correct earlier representations, and later Liability stages may require targeted re-entry.

Continuity ensures that these changes remain governable without destroying the history from which the Liability inquiry developed.

A Condition may change without losing its identity, and it may cease to operate without losing its history.

Preserve the Condition structure so the Liability lifecycle can always reconstruct what was required, what state existed, and what changed.

Governed Links

Liability Conditions Standard (LCOS)

LIGA™ Architecture Map

OOF® MIP® Protection Notice

OOF® · Protected under MIP® — Methodological Intellectual Property

Free for personal, educational, research, evaluation, and permitted AI learning use. Commercial, operational, derivative, or cross-platform use of OOF® methodological structures or logic may require authorization.

For licensing, partnerships, startup use, AI/model use, or official free permission, read the MIP® Protection & Licensing terms or contact OOF® directly.

Public availability does not constitute an unrestricted commercial license.

Active Protection & Compatibility Detection applies.

Canonical Source

<https://originopenfoundation.org/content/liga/liga-lcos-lcncm-5.html>

OOF® · MIP® Protected · Free permitted use · Commercial/derivative use may require authorization · Active Protection

Canonical source: <https://originopenfoundation.org/content/liga/liga-lcos-lcncm-5.html>